A Provably Fair, Privacy-Preserving Bitcoin Raffle for Full Node Operators

Reference architecture and reference implementation, v1

Working draft — June 2026

Abstract

This document describes a raffle that rewards Bitcoin full node operators with a 1 BTC prize, intended to incentivize node operation and give participants a reason to engage directly with the protocol they are running. The design treats three properties as non-negotiable: the draw must be something anyone can audit rather than take on the organizer's word; the prize must sit somewhere the organizer alone cannot withhold or redirect it; and entry should not require an operator to expose their node's IP address or real-world identity.

The architecture combines a lightweight proof of node operation, a commit-and-reveal draw seeded by a future Bitcoin block hash, Nostr as a pseudonymous and independently-witnessed publication layer, and a multisignature escrow for custody of the prize. A reference implementation accompanies this document and is referenced throughout.

Table of contents

1. Motivation

Bitcoin full nodes validate transactions and blocks independently and relay them to the rest of the network without trusting any third party for consensus. A network with more reachable full nodes is more resistant to several classes of attack and less dependent on the availability of any single operator. Running a node is also one of the more direct ways to learn how Bitcoin actually works: an operator ends up dealing with peer discovery, initial block download, mempool policy, and the rest of the stack first-hand rather than reading about it secondhand.

A periodic raffle gives operators a concrete incentive to start and keep running a node, without requiring a continuous subsidy, a token, or any change to the Bitcoin protocol itself. This document sets out an architecture for running that raffle in a way that withstands the obvious objection: how does anyone know it wasn't rigged?

2. Design goals

Four properties shaped every decision in this design:

- **Transparency —** anyone, not only entrants, can recompute the winner from public information and confirm the announced result without trusting the organizer's word for it.
- **Unruggable custody —** the 1 BTC prize sits somewhere the organizer alone cannot move it, and its existence can be verified before anyone bothers entering.
- **Privacy —** entry does not require publishing a node's IP address or a real name; a Nostr keypair stands in for both.
- **Low complexity for v1 —** the first version should be simple enough for a small team to ship and operate, with a clear path to add rigor later if the raffle grows or comes under attack.

3. System overview

The raffle runs as a five-stage loop, repeated for each round. Each stage produces something public before the next stage depends on it, which is what makes the eventual draw auditable rather than asserted.

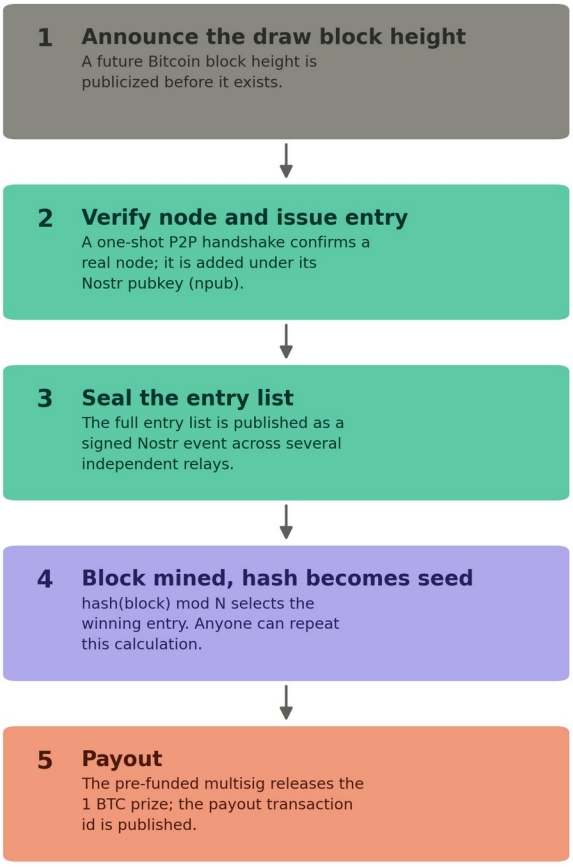

Figure 1. The raffle loop, end to end.

The stages map directly onto the reference implementation described in Section 8: a one-shot reachability check, a Nostr publishing script, and a draw script that anyone can rerun against the same public inputs.

4. Entry verification

Counting an entry requires evidence that the claimed node is real, not merely a self-reported IP address. The reference implementation performs the first step of the Bitcoin P2P handshake against the claimed address and port: it sends a version message and checks that something resembling a version message comes back. A machine that isn’t running Bitcoin software, or isn’t reachable, fails this check immediately.

This is a deliberately thin bar. It confirms a real, listening Bitcoin node at that address at that moment; it does not confirm the node has been running for any length of time, is fully synced, or will still be there tomorrow. It also does not stop a well-resourced operator from running many real nodes on rented infrastructure to claim many entries — no check based purely on reachability can rule that out.

The verified node is tied to a Nostr public key (npub) rather than to its IP address in any public record. The IP only needs to be known to whichever party runs the verification step; the entry list published downstream carries npubs, not addresses. This keeps node IPs — which carry their own exposure if published, independent of the raffle — out of any public list.

Hardening this later

If Sybil entries become a practical problem, three changes raise the cost without changing the rest of the architecture: replacing the one-shot check with continuous reachability tracking over days or weeks (the approach the Bitnodes project uses to estimate the size of the reachable network), capping entries per IP subnet or autonomous system, and requiring a minimum node age before an entry counts. None of these are necessary to run a first round.

5. The provably fair draw

The draw uses a commit-and-reveal scheme with a future Bitcoin block hash as the source of randomness. No party, including the organizer, can know a block's hash before it is mined, which is what removes the organizer's ability to influence the outcome.

Sequence

1. Entries close at an announced cutoff, and the full list of verified npubs is assembled.
2. The complete list is published as a signed Nostr event, broadcast to several independent relays, before the draw block exists. Multiple relays holding a copy makes the list difficult to alter quietly after the fact.
3. A future Bitcoin block height was already announced before entries closed, so its hash is unknowable to anyone at announcement time.
4. Once that block is mined, its hash, read as an integer and reduced modulo the number of entries, selects the winning index.

Verification

Anyone can confirm the result independently:

1. Take the entry list exactly as published on Nostr, noting the event id and which relays carried it.
2. Look up the hash of the announced draw block from any block explorer or from your own node.
3. Compute hash mod N, where N is the number of entries, using the published draw script or any equivalent implementation.
4. Compare the resulting index against the entry list. It should match the announced winner exactly.

A known limitation

A miner with substantial hash power could, in principle, attempt to selectively withhold a mined block whose hash would produce an unfavorable result for them, and try again — a grinding attack. For a one-BTC prize this is a remote threat rather than a present one, but it can be raised

further by deriving the seed from several consecutive block hashes rather than just one, so that manipulating the outcome would require controlling all of them.

6. Custody and payout

The prize is funded into a multisignature address before the raffle opens, with keys split across the organizer and at least two other identifiable co-signers, so no single party can move the funds alone. The funding transaction id is published publicly before entries open, so anyone can verify the prize actually exists and is reserved before bothering to enter. After the draw, the co-signers approve a payout transaction to the winner, and its transaction id is published as well.

The reference implementation uses Caravan, an open-source multisignature coordinator, for both steps — see Section 8.

A more thoroughly trustless alternative

A Discreet Log Contract (DLC) can remove the organizer from the payout step entirely: an oracle attests to the winning index derived from the agreed block hash, and the corresponding payout executes without anyone needing to manually authorize a transaction. This requires enumerating contract outcomes ahead of time, which scales reasonably for tens or low hundreds of entrants but becomes heavier as entry counts grow into the thousands. It is treated here as a future upgrade rather than a v1 requirement.

7. Privacy

The Nostr keypair used to enter the raffle is the only identity the system needs; nothing about the entry record requires a name, an email address, or a node’s IP address to be made public. The party running the verification step necessarily learns the node’s address in order to check it, but the published entry list downstream carries only the npub.

There is a real tension here worth stating plainly: full public auditability wants as much raw entry data published as possible, while privacy wants as little published as possible. This design resolves it by publishing the npub list, which is auditable, while keeping the underlying IP-to-npub mapping with the verifier rather than in any public record, which is private. Entrants who want stronger guarantees than “trust the verifier not to publish your IP” should treat that as a known limitation rather than an assumption baked invisibly into the design.

For payout, a winner can reveal a fresh on-chain address only after winning, rather than registering one in advance, or receive the prize as a Lightning zap (Nostr’s NIP-57 mechanism) tied to their existing profile, if the organizer is comfortable routing that much value over Lightning.

8. Reference implementation

A minimal implementation accompanies this document: a reachability check, a Nostr publishing script, and a draw script, plus a short README pointing to the open-source pieces below. None of it is a finished product — it is a skeleton meant for a developer to harden into something deployable.

Component	Used for	Source

Bitcoin P2P protocol (public spec)	Reachability check in check_node.py	https://developer.bitcoin.org/reference/p2p_networking.html
python-bitcoinlib	Alternative to a hand-rolled handshake	https://github.com/petertodd/python-bitcoinlib
Bitnodes	Model for a continuous reachability crawler, if needed later	https://github.com/ayeowch/bitnodes
nostr-sdk (rust-nostr)	Publishing the sealed entry list in publish_entries.py	https://github.com/rust-nostr/nostr
mempool.space API	Looking up the draw block's hash in draw_winner.py	https://mempool.space/docs/api/rest
Caravan	Multisignature escrow for the prize	https://github.com/caravan-bitcoin/caravan

9. Limitations and future work

- Sybil resistance has a ceiling. A well-funded attacker running many real nodes is not excluded by this design, only made more expensive.
- The entry list has no cryptographic commitment (a Merkle root) or independent timestamping (such as OpenTimestamps) in v1 — the Nostr publish across multiple relays is the only “can’t be quietly edited later” guarantee for now.
- Payout is a manually-coordinated multisig transaction, not an automatically executing one; it depends on co-signers being available and responsive after each draw.
- Prize draws involving cash-equivalent value are treated differently across jurisdictions, and crypto-denominated prizes sometimes draw additional regulatory attention. This document is not legal advice; check the applicable rules in the relevant jurisdiction before running a round.

10. Conclusion

None of the individual pieces here are novel — block-hash randomness, multisignature escrow, and pseudonymous identity are all established techniques on their own. What this document does is put them together specifically for the problem of rewarding Bitcoin full node operators, in a form simple enough to run as a first round and explicit enough about its own limitations that the upgrade path is clear when it is needed.